Open Access | https://doi.org/10.25080/fygp5764



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

PanelBox: A Comprehensive Python Library for Panel Data Econometrics

Gustavo Coelho Haase[1], **Paulo Henrique Dourado**[1], and **Osvaldo Candido**[2]

[1]Banco do Brasil S.A., Brasília, Brazil, [2]Graduate School in Economics, Universidade Católica de Brasília, Brasília, Brazil

Abstract

Panel data econometrics is ubiquitous in economic and social-science research, yet Python lacks a comprehensive library for advanced panel analysis: researchers have to fall back on proprietary Stata or on R, breaking otherwise Python-native, reproducible workflows. We introduce PanelBox, the first all-in-one Python library for panel data econometrics, implementing more than 70 models across 11 families — static linear models, dynamic GMM estimators (Arellano–Bond, Blundell–Bond with the Windmeijer correction), discrete choice, count data, spatial econometrics, stochastic frontier analysis, quantile regression, panel VAR/VECM, censored and selection models, and the first comprehensive Python implementation of panel unit-root and cointegration tests. The library adds 50+ diagnostic tests, 11 standard-error estimators, four bootstrap methods, an integrated HTML/LaTeX/Markdown reporting system with interactive Plotly visualizations, and 103 bundled datasets, all on top of the scientific Python stack (NumPy, pandas, SciPy, statsmodels). We validate PanelBox against R's `plm`, reproducing its difference-GMM, fixed-effects and panel unit-root results to three or more decimals on the Arellano–Bond data, and illustrate it with a canonical dynamic labor-demand application. With 3,900+ tests and an MIT license, PanelBox lets researchers run sophisticated panel analysis entirely in Python.

Keywords panel data, econometrics, dynamic panel GMM, spatial econometrics, reproducible research

1. Introduction

PanelBox did not begin as a general-purpose econometrics library. We work in model validation for credit risk, where econometric models estimated on panel data must be independently re-estimated, stress-tested, and documented before they are allowed into production. Our production systems run on Python; the reference implementations of the methods we needed did not. Every validation cycle meant exporting data to Stata or R, re-importing results, and reconciling discrepancies by hand — slow, error-prone, and impossible to automate end to end. What started as an internal toolkit for validating panel models in pure Python grew, model family by model family, into PanelBox.

The data at the center of a production econometrics workflow like ours are *panel data* — repeated observations on the same entities over time, such as firms observed annually or countries tracked across decades. Panels are the empirical backbone of modern economics and the social sciences [1], [2], and for a simple reason: every firm or country has stable characteristics that no dataset records — management quality, institutions, culture. A cross-sectional regression silently attributes the effect of these unobserved traits to the variables it does observe, biasing its conclusions. Because a panel follows each entity over time, it can compare each entity with itself, removing the influence of anything constant — the

Published Jul 13, 2026

Correspondence to
Gustavo Coelho Haase
gustavohaase@gmail.com

Open Access

Copyright © 2026 Haase *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

unobserved heterogeneity that panel methods are built to neutralize. Combined with the time dimension, this lets researchers estimate genuinely dynamic relationships, from labor-market adjustment [3] to production functions [4] and cross-country growth [5].

The gap we encountered, it turned out, is far wider than our use case: the tooling for these methods is unevenly distributed across computing environments. Stata's `xtabond2` command [6] is the de facto standard for dynamic panel GMM, and R offers the mature `plm` package [7] for linear panels and specialized packages such as `pdynmc` [8] for dynamic ones. Python — now dominant in data science and increasingly in quantitative social science thanks to NumPy [9], pandas [10], and SciPy [11] — has lagged conspicuously behind in panel econometrics.

Existing Python libraries cover only fragments of the workflow. `linearmodels` [12] provides static fixed- and random-effects models, the workhorse estimators that remove, or explicitly model, the stable entity traits discussed above. It also offers some support for instrumental variables (proxy regressors used in place of a variable that is correlated with the error term). It has no dynamic GMM, however, the estimator required when the outcome depends on its own past, which we introduce below. `pyfixest` excels at high-dimensional fixed effects but offers neither GMM nor dynamic panels; and `statsmodels` [13] has time-series tools but no panel-specific GMM. Critically, none of them implement panel unit-root or cointegration tests, which are routine in Stata (`xtunitroot`, `xtcointtest`) and R (`plm::purtest`). The consequence is a broken workflow: analysts either limit themselves to basic models in Python, or export their data to Stata or R, forfeiting reproducibility and integration with the rest of the Python stack.

PanelBox closes this gap. Version 1.0 implements more than 70 models across 11 families in a single package, validated numerically against established software. Beyond breadth, it contributes three things that are new to Python: a validated dynamic-GMM implementation with the Windmeijer finite-sample correction; the first comprehensive panel unit-root and cointegration testing suite; and an intelligent algorithm for unbalanced panels that retains far more data than naive complete-case approaches. The library is open source (MIT), integrates natively with pandas, and ships with interactive visualization, publication-ready reporting, and 103 datasets for immediate experimentation. The paper follows the workflow a researcher actually traverses — specify and estimate a model, probe its assumptions, and report the results — with the depth reserved for the three novel contributions; Breadth: eleven model families summarizes the full catalogue, and Using and extending PanelBox describes how the library is used and extended in practice.

2. Library design: one workflow from data to report

PanelBox is designed to model the standard estimation workflow of applied econometrics. The user starts from a pandas `DataFrame` in long format; a common data layer wraps it together with entity and time identifiers, and everything downstream consumes that layer. Estimators accept R-style formulas via `patsy` and return a result object that follows the conventional `statsmodels` API — `summary()`, `conf_int()`, `to_latex()`, `to_html()` — so that the same result flows into the shared inference layer (11 standard-error types, four bootstrap methods), the diagnostics layer (50+ tests), and the reporting layer that renders self-contained HTML, LaTeX, or Markdown. Figure 1 summarizes this pipeline.

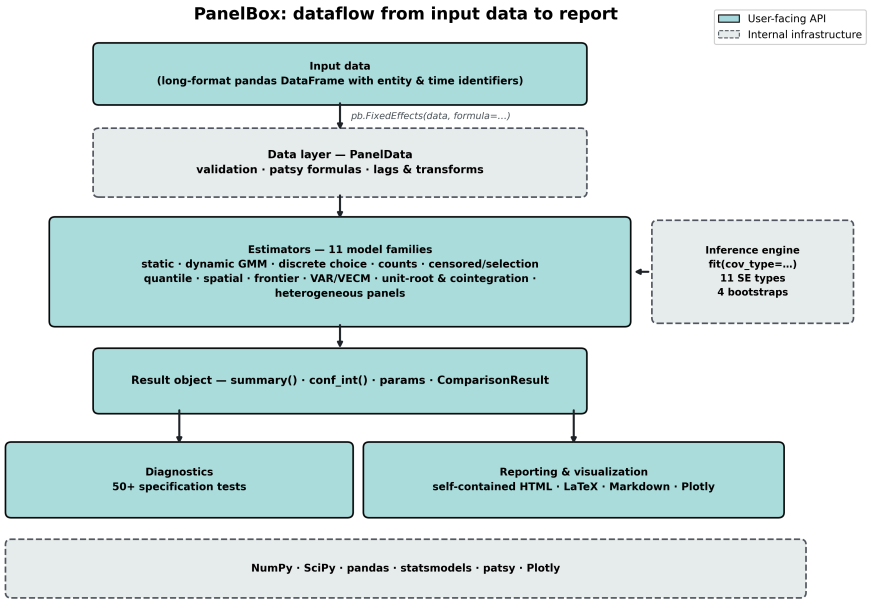


Figure 1. *PanelBox as a dataflow. Input data, represented as a long-format pandas `DataFrame`, enter the shared data layer, flow through the 11 estimator families — with the inference engine supplying standard errors and bootstraps at `fit()` time — and the resulting objects feed the diagnostics and reporting layers. Solid colored boxes are the user-facing API; dashed gray boxes are internal infrastructure.*

Because the API follows conventions the community already knows — `statsmodels`-style formulas and result objects, with an sklearn-like construct-then-fit pattern — a minimal estimation holds no surprises:

```
import panelbox as pb

data = pb.load_dataset("abdata")
fe = pb.FixedEffects(
    "n ~ w + k", data,
    entity_col="id", time_col="year",
)
result = fe.fit(cov_type="clustered")
print(result.summary())
```

The workflow's entry point is the static linear family, which implements pooled OLS, fixed effects (within), random effects (feasible GLS), the between estimator, and first differences for the model

$$y_{it} = \boldsymbol{x}_{it}'\boldsymbol{\beta} + \alpha_i + \lambda_t + \varepsilon_{it}, \qquad i = 1, \ldots, N,\ t = 1, \ldots, T, \tag{1}$$

where α_i are entity effects — absorbing everything specific to entity i that is constant over time, such as a firm's management quality — and λ_t are time effects, absorbing shocks common to all entities in period t, such as a recession year. Fixed effects are computed by within-transformation using sparse matrix operations and pandas `groupby`, so the estimator scales to panels with thousands of entities and tens of thousands of observations (the benchmarks in Figure 4 go up to $N = 2{,}500$) and supports unbalanced panels with entity-specific T_i.

3. Estimating dynamics: panel GMM

Many economic outcomes depend on their own past. Employment adjusts slowly because hiring and firing are costly; this year's GDP is anchored to last year's. Capturing such persistence means adding the lagged outcome $y_{i,t-1}$ as a regressor:

$$y_{it} = \alpha\, y_{i,t-1} + \boldsymbol{x}_{it}'\boldsymbol{\beta} + \eta_i + \varepsilon_{it}. \tag{2}$$

That innocent-looking addition breaks both estimators of the previous section. The lagged outcome is itself driven by the entity effect η_i, so it is correlated with the composite error: pooled OLS is biased upward, and the within estimator downward (the Nickell bias, S. Nickell [14]) — the true dynamics lie somewhere in between, and neither estimator can find them. The resolution, proposed by M. Arellano and S. Bond [3] and R. Blundell and S. Bond [4], is to transform away η_i and instrument the problematic regressor with its own older lags: old enough to be uncorrelated with today's shock, recent enough to remain informative. This machinery — dynamic panel GMM — is PanelBox's flagship capability, and previously had no validated Python implementation.

Difference GMM [3] first-differences (2) to remove η_i and exploits the moment conditions $\mathrm{E}\big[y_{i,t-s}\,\Delta\varepsilon_{it}\big] = 0$ for $s \geq 2$, so lagged levels instrument the differenced regressors. **System GMM** [4] augments the differenced equations with the level equations, instrumenting them with lagged differences; this restores efficiency and reduces finite-sample bias when α is close to unity and lagged levels are weak instruments. PanelBox implements both, in one-step and two-step variants — one-step GMM weights the moment conditions with a fixed matrix, while two-step re-estimates the weights from the first step's residuals to gain asymptotic efficiency — and generates the instrument sets automatically:

```
gmm = pb.SystemGMM(
    data, dep_var="n", lags=1,
    id_var="id", time_var="year",
    exog_vars=["w", "k"],
    collapse=True, two_step=True,
    time_dummies=False,
)
res = gmm.fit()
print(f"Hansen J p-value: {res.hansen_j.pvalue:.3f}")
print(f"AR(2) p-value:    {res.ar2_test.pvalue:.3f}")
```

Windmeijer correction. Two-step GMM is asymptotically efficient but its standard errors are severely biased downward in finite samples — by 30–80% in typical applications. F. Windmeijer [15] derives a finite-sample correction that accounts for the estimated weight matrix; PanelBox applies it by default, following the algorithm in D. Roodman [6].

Instrument proliferation. Because the instrument count grows quadratically in T, it can exceed N and overfit the endogenous regressors [16]. PanelBox warns automatically when the instrument count approaches N and implements the `collapse` option, which combines the per-period instruments for each variable into a single column while preserving the moment conditions.

3.1. Intelligent handling of unbalanced panels

Missing observations are a recurring practical obstacle in dynamic GMM: because each instrument depends on a specific lag (e.g. $y_{i,t-2}$ instruments Δy_{it}), a single gap can invalidate instruments. The common practical response is blunt: many practitioners drop every entity that is not observed in every period (complete-case analysis), causing severe sample loss.

PanelBox instead validates instruments observation-by-observation. For each entity i and period t it (1) identifies the required instrument lags, (2) checks their availability, (3) builds

an entity-specific instrument set from whatever lags exist, and (4) assembles the block-diagonal instrument matrix $\boldsymbol{Z}$ from variable-sized blocks (Figure 2). An observation is kept whenever valid instruments exist, rather than discarding the whole entity. The Arellano–Bond employment data illustrate the difference: 140 firms are observed for 7 to 9 years each (1,031 observations), and only 14 of them for all 9 years. Complete-case analysis keeps those 14 firms and estimates the differenced equation on 98 observations, 9.5% of the panel. PanelBox estimates it on 751 observations (72.8%), every observation for which a valid instrument exists once one lag and one difference are consumed.

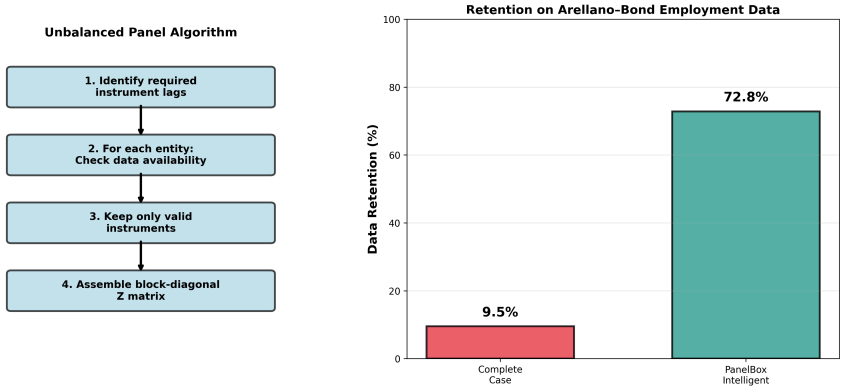


Figure 2. *The unbalanced-panel algorithm: instruments are validated per observation and entity-specific instrument sets are assembled into a block-diagonal matrix. On the Arellano–Bond employment data this retains 72.8% of observations against 9.5% for complete-case analysis.*

4. Testing assumptions: unit roots and cointegration

A real-world econometrics workflow does not end when estimation is done: the validity of the estimates rests on assumptions that must themselves be tested. For macroeconomic panels the first question is whether each variable has a stable long-run level or wanders without one (a *unit root*): regressing one wandering series on another produces convincing-looking but spurious correlations, so testing for unit roots — and, when variables do wander, for a genuine shared long-run relationship (*cointegration*) — is the standard safety check before estimating relationships among variables such as GDP, prices, or exchange rates. This capability is unique to PanelBox among Python panel libraries, despite being standard in macroeconomics and finance.

PanelBox implements the three standard panel unit-root tests and the two standard cointegration tests. The unit-root tests build on the entity-specific augmented Dickey–Fuller regression, testing $H_0 : \rho_i = 0$. The LLC test [17] assumes a common ρ and uses a pooled adjusted t-statistic; the IPS test [18] allows heterogeneous ρ_i and averages the individual t-statistics into an asymptotically normal W; and Fisher-type tests [19] combine the per-entity p-values. For $I(1)$ variables, the Pedroni [20], [21] and Kao [22] tests assess cointegration; Pedroni reports seven statistics and resolves them by majority vote.

```
from panelbox.validation.unit_root import LLCTest, IPSTest

llc = LLCTest(data, variable="log_gdp",
              entity_col="country", time_col="year", trend="ct")
print(llc.run().pvalue)
```

Together these enable a complete workflow — test for unit roots, test for cointegration when variables are $I(1)$, then estimate the appropriate model — illustrated in the growth application of Application: dynamic labor demand.

5. Breadth: eleven model families

Linear panels are only part of applied practice: economic data routinely involve binary outcomes, counts, censoring, spatial dependence, and efficiency frontiers. PanelBox covers these in the same interface; Table 1 maps each family to its closest Stata and R equivalents, and we highlight below only the capabilities not available elsewhere in Python. The complete catalogue, with an executable example notebook per family, is in the official documentation at https://panelbox.readthedocs.io/.

Table 1. *The 11 model families in PanelBox and their closest equivalents in Stata and R. A dash means no established equivalent.*

Family	Representative estimators in PanelBox	Stata	R
Static linear	Pooled OLS, FE, RE, between, first differences	`xtreg`	`plm`
Dynamic GMM	Difference/system GMM, Windmeijer correction, Anderson–Hsiao, LSDVC	`xtabond2`	`plm::pgmm`, `pdynmc`
Unit-root & cointegration tests	LLC, IPS, Fisher; Pedroni, Kao	`xtunitroot`, `xtcointtest`	`plm::purtest`
Discrete choice	Pooled/FE/RE logit & probit, multinomial, ordered, dynamic binary	`xtlogit`, `xtprobit`	`bife`, `pglm`
Count data	Poisson (pooled/FE/QML), negative binomial, zero-inflated, PPML	`xtpoisson`, `ppmlhdfe`	`pglm`, `fixest`
Censored & selection	Panel tobit, sample-selection models	`xttobit`, `xtheckman`	`censReg`
Quantile regression	Pooled, FE, Canay, Machado–Santos Silva	`xtqreg`	`rqpd`
Spatial panels	SAR, SEM, SDM, GNS, dynamic spatial	`spxtregress`	`splm`
Stochastic frontier	Battese–Coelli, Greene TFE/TRE, four-component	`sfpanel`	`frontier`
Panel VAR/VECM	IRFs, FEVD, Granger causality	`pvar`	`panelvar`
Heterogeneous panels	Mean-group estimators, SUR	`xtmg`, `sureg`	`plm::pmg`, `systemfit`

Several entries go beyond their Stata/R counterparts. The discrete-choice family includes dynamic binary models with the J. M. Wooldridge [23] initial-conditions correction and conditional maximum-likelihood fixed effects [24], with average and representative-value marginal effects computed with delta-method standard errors; the count family includes the PPML estimator [25] widely used for gravity models. The spatial family estimates five models by quasi-maximum likelihood [26] and decomposes results into direct and indirect (spillover) effects via the spatial multiplier $(\boldsymbol{I}_N - \rho \boldsymbol{W})^{-1}\beta_k$. The frontier family implements the S. C. Kumbhakar, G. Lien, and J. B. Hardaker [27] four-component model separating *persistent* from *transient* inefficiency [28], [29], [30], and the quantile family includes the J.

A. F. Machado and J. M. C. Santos Silva [31] location-scale estimator, which guarantees non-crossing quantiles [32], [33] — all three previously unavailable in Python.

6. Inference, diagnostics, and reporting

Once a model is estimated, the workflow turns to defending it. All standard errors use the sandwich form $\widehat{\mathrm{Var}}\left(\hat{\beta}\right) = (\boldsymbol{X}'\boldsymbol{X})^{-1}\,\hat{\boldsymbol{\Omega}}\,(\boldsymbol{X}'\boldsymbol{X})^{-1}$, differing in the inner "meat" matrix $\hat{\boldsymbol{\Omega}}$ (the term of art for the filling of the sandwich form): HC0–HC3 for heteroskedasticity, one-way and two-way clustering, and the Driscoll–Kraay and Newey–West HAC estimators. Four bootstraps — pairs, wild, block, and residual — cover small-sample and non-standard cases.

The diagnostic suite includes the Hausman test [34] comparing FE and RE, and the GMM trio that governs dynamic-panel validity: the Hansen J test of overidentifying restrictions [35], and the Arellano–Bond AR(1) and AR(2) tests for serial correlation, where AR(1) is expected to reject and AR(2) to not reject.

Result objects render publication-ready output. `ComparisonResult` aligns several models side by side, and the reporting layer produces self-contained HTML with embedded interactive Plotly charts (28+ chart types across residual diagnostics, model comparison, and econometric tests) [36], or static LaTeX/Markdown:

```
comparison = pb.ComparisonResult({
    "Pooled OLS": ols_result, "FE": fe_result,
    "Diff-GMM": diffgmm_result, "Sys-GMM": sysgmm_result,
})
print(comparison.summary())
```

7. Using and extending PanelBox

PanelBox makes no assumptions about where data come from. Any long-format pandas `DataFrame` with an entity column and a time column works directly — loaded from CSV, Parquet, SQL, or an API via the usual pandas readers — and unbalanced panels are handled natively, so no reshaping or gap-filling is required. The 103 bundled datasets are conveniences for teaching, benchmarking, and replication, not a requirement.

All estimators share one contract: construct with the data and a specification, call `fit()`, receive a results object with the same methods everywhere. Comparing a fixed-effects model with a system-GMM model is a one-line change of class, and `ComparisonResult` aligns any set of results. Within this contract, the supported customization points are the covariance estimator and bootstrap scheme (per `fit()` call), the instrument design for GMM (`gmm_max_lag`, `collapse`), arbitrary variable transformations through `patsy` formulas, marginal-effects options for nonlinear models, and the output format of every report (HTML, LaTeX, Markdown).

At the moment there are two limitations in PanelBox that we hope to overcome in the future. First, there is no public API for user-defined moment conditions or custom estimators: internally all models subclass a common base model and results class, and a documented, stable extension interface is on the roadmap. Second, GMM is restricted to linear dynamic panels. The official documentation (https://panelbox.readthedocs.io/) includes a gallery of executable Jupyter notebooks covering every model family end to end, which is the recommended starting point for adapting the library to a new use case.

8. Application: dynamic labor demand

To illustrate the capabilities of the library, we use it to reproduce the canonical application of M. Arellano and S. Bond [3]: a dynamic labor-demand equation on UK firm-level data, an unbalanced panel of $N = 140$ firms observed for 7 to 9 years between 1976 and 1984 (1,031 observations). The model

$$n_{it} = \alpha\, n_{i,t-1} + \beta_w w_{it} + \beta_k k_{it} + \eta_i + \lambda_t + \varepsilon_{it} \tag{3}$$

relates log employment n_{it} to log real wages w_{it} and log capital k_{it}; α captures employment adjustment costs. We estimate it four ways to expose the bias–consistency trade-off:

```
data = pb.load_dataset("abdata")
static = data.dropna(subset=["nL1"])  # nL1: lagged employment
ols = pb.PooledOLS(
    "n ~ nL1 + w + k", static, entity_col="id", time_col="year",
).fit()
fe = pb.FixedEffects(
    "n ~ nL1 + w + k", static, entity_col="id", time_col="year",
).fit(cov_type="clustered")
gmm_spec = dict(
    dep_var="n", lags=1, id_var="id", time_var="year",
    exog_vars=["w", "k"], gmm_max_lag=9, collapse=True,
    two_step=True, time_dummies=True,
)
diffgmm = pb.DifferenceGMM(data, **gmm_spec).fit()
sysgmm = pb.SystemGMM(data, **gmm_spec).fit()
```

The estimates follow the bracketing argument of S. R. Bond [37] (Figure 3). Pooled OLS yields $\hat{\alpha} = 0.931$, biased upward by the correlation between $n_{i,t-1}$ and η_i; fixed effects gives $\hat{\alpha} = 0.528$, biased downward by the Nickell bias. A consistent estimate should lie between the two, and both GMM estimators do: difference GMM at $\hat{\alpha} = 0.877$ (standard error 0.234) and system GMM at 0.563 (0.194). The wide confidence intervals are the price of instrumenting a persistent regressor with its own lags; the system estimator's additional level moment conditions reduce the standard error but, with only 140 firms and collapsed instruments, do not eliminate it. The wage elasticity is negative in every specification ($\hat{\beta}_w$ between -0.25 and -0.50 outside OLS) and capital enters positively. The diagnostics do not reject either GMM specification: the Hansen J test gives $p = 0.244$ (difference) and $p = 0.230$ (system), AR(1) rejects as expected ($p < 0.01$) while AR(2) does not ($p = 0.57$ and 0.87), and the instrument counts (16 and 18) stay far below the 140 entities.

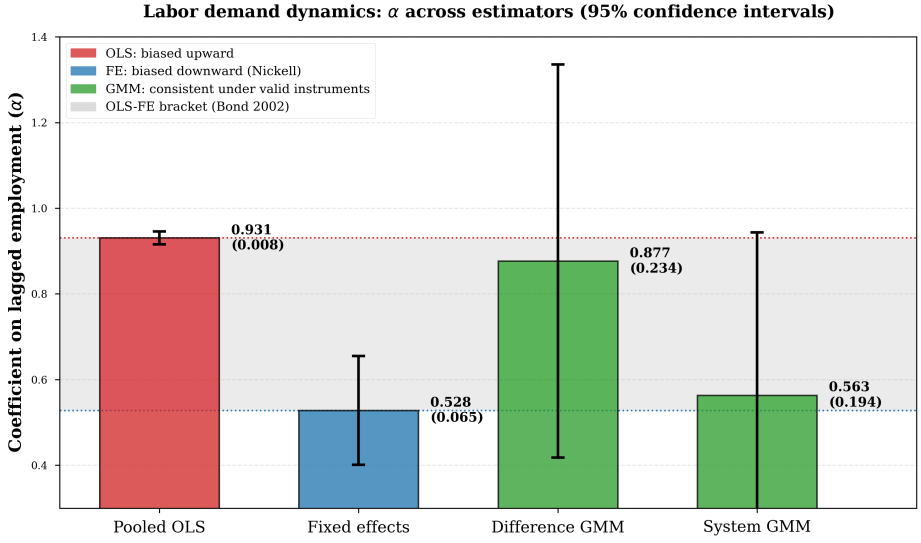


Figure 3. *Coefficient on lagged employment across pooled OLS, fixed effects, difference GMM, and system GMM, with 95% confidence intervals. The shaded band is the OLS–FE bracket of S. R. Bond [37]: OLS is biased upward and fixed effects downward, so a consistent estimate is expected to fall between them, as both GMM estimates do.*

The same building blocks support a full nonstationary-panel workflow on the Penn World Table 10.01 [38], using the 129 countries with complete data for 1970–2019. The IPS test does not reject a unit root in log GDP per capita ($p = 1.00$) while rejecting it in first differences ($p < 0.001$), so the series is treated as $I(1)$; the LLC test rejects in levels, a known consequence of its common-root assumption under cross-sectional dependence, and the disagreement itself is informative. Pedroni's test then rejects no-cointegration between log GDP per capita, the investment share and human capital on five of seven statistics. A conditional-convergence regression on non-overlapping five-year panels gives an autoregressive coefficient of 0.944 (0.006) by pooled OLS and 0.791 (0.027) by fixed effects; system GMM returns 0.808 (0.050), inside the bracket and implying convergence at roughly 4% per year, in line with panel estimates in the growth literature [39]. The Hansen test rejects ($p = 0.002$), however, flagging the instrument validity problems that are well known for this specification and that the library's diagnostics are designed to expose.

9. Validation and performance

We validate PanelBox against R's `plm` [7], estimating identical specifications on the Arellano–Bond employment data and computing relative differences $\Delta = |\,\mathrm{PanelBox} - \mathrm{reference}|\,/\,|\,\mathrm{reference}|$ (Table 2). For difference GMM — two-step, collapsed instruments, year dummies, Windmeijer-corrected standard errors — every coefficient, standard error, and the Hansen J statistic match `plm::pgmm` to at least three decimals, and the instrument count is identical. Fixed-effects estimates match to six decimals. The panel unit-root tests were compared with `plm::purtest` on log GDP per capita from the Penn World Table: the IPS W statistic is identical, and the LLC statistic agrees within 2%, the residual difference coming from the long-run variance kernel.

System GMM requires a different reference. `plm` and Stata's `xtabond2` [6] weight the level equations differently and do not agree with each other on this estimator; PanelBox follows the `xtabond2` convention (its default H matrix, which links the differenced and level equations). We therefore compare system GMM with `pydynpd` [40], a Python implementation of the `xtabond2` algorithm validated against Stata by its author: coefficients agree within 1–2% and the Hansen J test is identical, the remaining gap being one level-equation observation per entity that `xtabond2` retains and PanelBox currently drops. A direct comparison with Stata is planned.

Table 2. *Numerical validation on the Arellano–Bond employment data (two-step GMM, collapsed instruments, year dummies; standard errors in parentheses). Difference GMM and fixed effects are compared with R plm; system GMM with pydynpd, a Python replica of Stata's xtabond2; unit-root tests with plm on Penn World Table log GDP per capita.*

Quantity	Reference	PanelBox	Δ (%)
Diff-GMM `L.n` (s.e.)	0.8766 (0.2342)	0.8766 (0.2342)	0.00
Diff-GMM `w` (s.e.)	-0.4587 (0.1698)	-0.4587 (0.1698)	0.00
Diff-GMM `k` (s.e.)	0.1907 (0.0649)	0.1907 (0.0649)	0.00
Diff-GMM Hansen J (p) / instruments	7.91 (0.244) / 16	7.91 (0.244) / 16	0.00
Sys-GMM `L.n` (s.e.)	0.5622 (0.1871)	0.5634 (0.1938)	0.21
Sys-GMM `w` (s.e.)	-0.2414 (0.1116)	-0.2467 (0.1107)	2.2
Sys-GMM `k` (s.e.)	0.3547 (0.1462)	0.3534 (0.1503)	0.37

Sys-GMM Hansen J (p) / instruments	9.33 (0.230) / 18	9.33 (0.230) / 18	0.00
FE `w` (s.e.)	-0.367774 (0.052323)	-0.367774 (0.052323)	0.00
FE `k` (s.e.)	0.640367 (0.020142)	0.640367 (0.020142)	0.00
IPS W (lags 1)	6.392	6.392	0.00
LLC t^* (lags 1)	-3.547	-3.518	0.82

PanelBox is implemented in pure Python on NumPy/SciPy. On simulated dynamic panels with $T = 10$, two-step system GMM takes 1.1 s for $N = 500$ and 4.6 s for $N = 2,500$, against 0.5 s and 3.1 s for `plm::pgmm` (Figure 4): within a factor of 1.5–2 of the compiled R implementation for $N \geq 500$, with a fixed overhead of about 0.6 s that dominates on small panels. Memory scales linearly in NT, and bootstrap inference parallelizes with near-linear speedup.

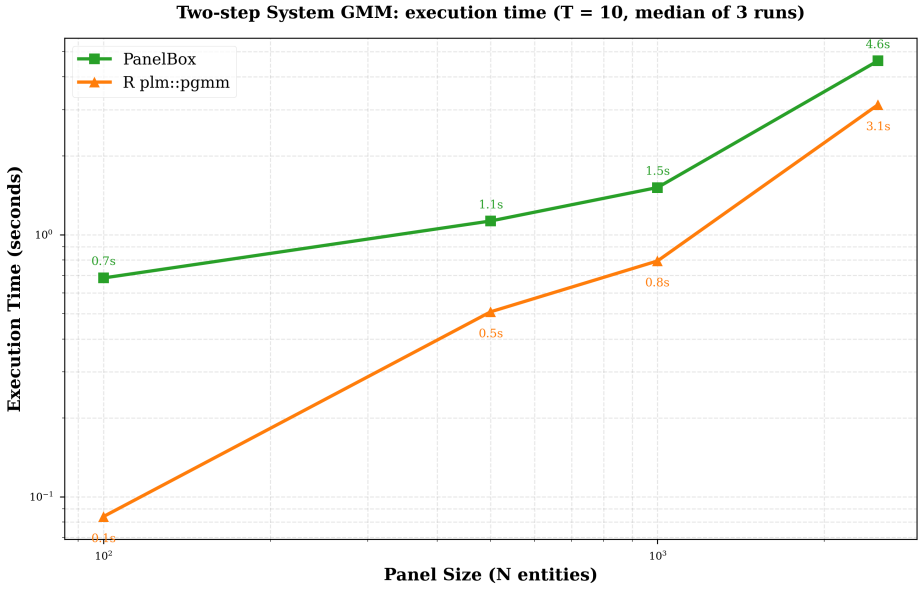


Figure 4. *Execution time of two-step system GMM as a function of the number of entities ($T = 10$, median of three runs): PanelBox versus R `plm::pgmm`.*

10. Conclusion

PanelBox brings comprehensive panel data econometrics to Python: 70+ models across 11 families, 50+ diagnostic tests, 11 standard-error estimators, an interactive visualization and reporting system, and 103 bundled datasets, in a single MIT-licensed package built on the scientific Python stack. Its difference GMM reproduces R's `plm` to three decimals and its system GMM follows the `xtabond2` conventions, it adds an unbalanced-panel algorithm that estimates on 72.8% of the Arellano–Bond observations where complete-case analysis uses 9.5%, and it offers capabilities — panel unit-root and cointegration tests, four-component stochastic frontiers, non-crossing quantile regression, panel VAR/VECM — previously unavailable in Python. The unusual emphasis on numerical validation throughout this paper is not incidental: it reflects the library's origin in production model validation, where matching the reference implementation is the requirement, not a nicety.

Current limitations include spatial-model scalability beyond $N \approx 2,000$ (mitigated by Chebyshev approximation), GMM restricted to linear dynamic panels, and no GPU acceleration. Planned work includes FMOLS/DOLS for cointegrated panels, double/debiased machine learning for panel data, GPU acceleration via JAX/CuPy, and Bayesian panel models. By eliminating the need to leave Python for sophisticated panel analysis, PanelBox makes

empirical economic research more reproducible, more integrated with the machine-learning ecosystem, and more accessible. It is available on PyPI (`pip install panelbox`) and on GitHub at https://github.com/PanelBox-Econometrics-Model/panelbox [41], with documentation, tutorials, example notebooks for every model family, and full replication materials for this paper.

11. Generative AI disclosure

Portions of this work were assisted using generative AI tools, specifically Anthropic's Claude. The tools were used to help draft and refine the prose of the manuscript, to improve clarity and phrasing, to suggest edits to the LaTeX/MyST markup, and to suggest code for parts of the PanelBox library and its replication scripts. Generative AI was **not** used to produce the numerical results, statistical validations (e.g., the comparisons against `xtabond2` and `plm`), or references reported here: these derive from actual executions of the software and were checked against the reference tools. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content, including all technical claims, results, and references.

References

[1] B. H. Baltagi, *Econometric Analysis of Panel Data*, 6th ed. Cham, Switzerland: Springer, 2021. doi: 10.1007/978-3-030-53953-5.

[2] J. M. Wooldridge, *Econometric Analysis of Cross Section and Panel Data*, 2nd ed. Cambridge, MA: MIT Press, 2010.

[3] M. Arellano and S. Bond, "Some Tests of Specification for Panel Data: Monte Carlo Evidence and an Application to Employment Equations," *The Review of Economic Studies*, vol. 58, no. 2, pp. 277–297, 1991, doi: 10.2307/2297968.

[4] R. Blundell and S. Bond, "Initial Conditions and Moment Restrictions in Dynamic Panel Data Models," *Journal of Econometrics*, vol. 87, no. 1, pp. 115–143, 1998, doi: 10.1016/S0304-4076(98)00009-8.

[5] S. R. Bond, A. Hoeffler, and J. Temple, "GMM Estimation of Empirical Growth Models," *Economics Papers 2001-W21*, 2001.

[6] D. Roodman, "How to Do xtabond2: An Introduction to Difference and System GMM in Stata," *The Stata Journal*, vol. 9, no. 1, pp. 86–136, 2009, doi: 10.1177/1536867X0900900106.

[7] Y. Croissant and G. Millo, "Panel Data Econometrics in \proglangR: The \pkgplm Package," *Journal of Statistical Software*, vol. 27, no. 2, pp. 1–43, 2008, doi: 10.18637/jss.v027.i02.

[8] M. Fritsch, A. A. Pua, and J. Schnurbus, "\pkgpdynmc: A Package for Estimating Linear Dynamic Panel Data Models Based on Nonlinear Moment Conditions," *Working Paper*, 2019.

[9] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, "Array Programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[10] W. McKinney, "\pkgpandas: A Foundational \proglangPython Library for Data Analysis and Statistics," in *Python for High Performance and Scientific Computing*, 2011, pp. 1–9.

[11] P. Virtanen, R. Gommers, T. E. Oliphant, and others, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[12] K. Baum, "\pkglinearmodels: Linear Models with Panel Data." [Online]. Available: https://github.com/bashtage/linearmodels

[13] S. Seabold and J. Perktold, "\pkgstatsmodels: Econometric and Statistical Modeling with \proglangPython," *Proceedings of the 9th Python in Science Conference*, pp. 92–96, 2010, doi: 10.25080/Majora-92bf1922-011.

[14] S. Nickell, "Biases in Dynamic Models with Fixed Effects," *Econometrica*, vol. 49, no. 6, pp. 1417–1426, 1981, doi: 10.2307/1911408.

[15] F. Windmeijer, "A Finite Sample Correction for the Variance of Linear Efficient Two-Step GMM Estimators," *Journal of Econometrics*, vol. 126, no. 1, pp. 25–51, 2005, doi: 10.1016/j.jeconom.2004.02.005.

[16] D. Roodman, "A Note on the Theme of Too Many Instruments," *Oxford Bulletin of Economics and Statistics*, vol. 71, no. 1, pp. 135–158, 2009, doi: 10.1111/j.1468-0084.2008.00542.x.

[17] A. Levin, C.-F. Lin, and C.-S. J. Chu, "Unit Root Tests in Panel Data: Asymptotic and Finite-Sample Properties," *Journal of Econometrics*, vol. 108, no. 1, pp. 1–24, 2002, doi: 10.1016/S0304-4076(01)00098-7.

[18] K. S. Im, M. H. Pesaran, and Y. Shin, "Testing for Unit Roots in Heterogeneous Panels," *Journal of Econometrics*, vol. 115, no. 1, pp. 53–74, 2003, doi: 10.1016/S0304-4076(03)00092-7.

[19] G. S. Maddala and S. Wu, "A Comparative Study of Unit Root Tests with Panel Data and a New Simple Test," *Oxford Bulletin of Economics and Statistics*, vol. 61, no. S1, pp. 631–652, 1999, doi: 10.1111/1468-0084.0610s1631.

[20] P. Pedroni, "Critical Values for Cointegration Tests in Heterogeneous Panels with Multiple Regressors," *Oxford Bulletin of Economics and Statistics*, vol. 61, no. S1, pp. 653–670, 1999, doi: 10.1111/1468-0084.0610s1653.

[21] P. Pedroni, "Panel Cointegration: Asymptotic and Finite Sample Properties of Pooled Time Series Tests with an Application to the PPP Hypothesis," *Econometric Theory*, vol. 20, no. 3, pp. 597–625, 2004, doi: 10.1017/S0266466604203073.

[22] C. Kao, "Spurious Regression and Residual-Based Tests for Cointegration in Panel Data," *Journal of Econometrics*, vol. 90, no. 1, pp. 1–44, 1999, doi: 10.1016/S0304-4076(98)00023-2.

[23] J. M. Wooldridge, "Simple Solutions to the Initial Conditions Problem in Dynamic, Nonlinear Panel Data Models with Unobserved Heterogeneity," *Journal of Applied Econometrics*, vol. 20, no. 1, pp. 39–54, 2005, doi: 10.1002/jae.770.

[24] G. Chamberlain, "Analysis of Covariance with Qualitative Data," *The Review of Economic Studies*, vol. 47, no. 1, pp. 225–238, 1980, doi: 10.2307/2297110.

[25] J. M. C. Santos Silva and S. Tenreyro, "The Log of Gravity," *The Review of Economics and Statistics*, vol. 88, no. 4, pp. 641–658, 2006, doi: 10.1162/rest.88.4.641.

[26] L.-f. Lee and J. Yu, "Estimation of Spatial Autoregressive Panel Data Models with Fixed Effects," *Journal of Econometrics*, vol. 154, no. 2, pp. 165–185, 2010, doi: 10.1016/j.jeconom.2009.08.001.

[27] S. C. Kumbhakar, G. Lien, and J. B. Hardaker, "Technical Efficiency in Competing Panel Data Models: A Study of Norwegian Grain Farming," *Journal of Productivity Analysis*, vol. 41, no. 2, pp. 321–337, 2014, doi: 10.1007/s11123-012-0303-1.

[28] G. E. Battese and T. J. Coelli, "Frontier Production Functions, Technical Efficiency and Panel Data: With Application to Paddy Farmers in India," *Journal of Productivity Analysis*, vol. 3, no. 1–2, pp. 153–169, 1992, doi: 10.1007/BF00158774.

[29] G. E. Battese and T. J. Coelli, "A Model for Technical Inefficiency Effects in a Stochastic Frontier Production Function for Panel Data," *Empirical Economics*, vol. 20, no. 2, pp. 325–332, 1995, doi: 10.1007/BF01205442.

[30] W. Greene, "Reconsidering Heterogeneity in Panel Data Estimators of the Stochastic Frontier Model," *Journal of Econometrics*, vol. 126, no. 2, pp. 269–303, 2005, doi: 10.1016/j.jeconom.2004.05.003.

[31] J. A. F. Machado and J. M. C. Santos Silva, "Quantiles via Moments," *Journal of Econometrics*, vol. 213, no. 1, pp. 145–173, 2019, doi: 10.1016/j.jeconom.2019.04.009.

[32] R. Koenker, "Quantile Regression for Longitudinal Data," *Journal of Multivariate Analysis*, vol. 91, no. 1, pp. 74–89, 2004, doi: 10.1016/j.jmva.2004.05.006.

[33] I. A. Canay, "A Simple Approach to Quantile Regression for Panel Data," *The Econometrics Journal*, vol. 14, no. 3, pp. 368–386, 2011, doi: 10.1111/j.1368-423X.2011.00349.x.

[34] J. A. Hausman, "Specification Tests in Econometrics," *Econometrica*, vol. 46, no. 6, pp. 1251–1271, 1978, doi: 10.2307/1913827.

[35] L. P. Hansen, "Large Sample Properties of Generalized Method of Moments Estimators," *Econometrica*, vol. 50, no. 4, pp. 1029–1054, 1982, doi: 10.2307/1912775.

[36] Plotly Technologies Inc., "Collaborative data science." [Online]. Available: https://plot.ly/

[37] S. R. Bond, "Dynamic panel data models: a guide to micro data methods and practice," *Portuguese Economic Journal*, vol. 1, no. 2, pp. 141–162, 2002, doi: 10.1007/s10258-002-0009-9.

[38] R. C. Feenstra, R. Inklaar, and M. P. Timmer, "The next generation of the Penn World Table," *American Economic Review*, vol. 105, no. 10, pp. 3150–3182, 2015, doi: 10.1257/aer.20130954.

[39] N. G. Mankiw, D. Romer, and D. N. Weil, "A Contribution to the Empirics of Economic Growth," *The Quarterly Journal of Economics*, vol. 107, no. 2, pp. 407–437, 1992, doi: 10.2307/2118477.

[40] D. Wu, "pydynpd: A Python package for dynamic panel model," *Journal of Open Source Software*, vol. 8, no. 83, p. 4416, 2023, doi: 10.21105/joss.04416.

[41] G. C. Haase, P. H. Dourado, and O. Candido, "PanelBox: A Comprehensive Python Library for Panel Data Econometrics." [Online]. Available: https://github.com/PanelBox-Econometrics-Model/panelbox